

Particle Interpolation and Mover Performance Analysis

March 10, 2026

Name	Roll Number
Tanuj Kanani	202301474
Chavda Mihirsinh	202301479

Abstract

This report presents a comprehensive performance analysis of a **bilinear interpolation algorithm** coupled with a newly introduced **particle Mover operation**, benchmarked across two distinct hardware platforms: a consumer-grade Lab PC (Intel Core i5-12500) and an HPC Cluster (Intel Xeon E5-2620 v3). Three structured experiments were conducted: **Experiment 01** evaluated how interpolation execution time scales with particle count (10^2 to 10^9) across three grid configurations on both platforms; **Experiment 02** assessed consistency of interpolation time across grid configurations with a fixed particle count of 10^8 ; and **Experiment 03** introduced the Mover phase and parallelized it using OpenMP with 4 threads. Results confirm $O(N)$ algorithmic complexity and demonstrate that the algorithm is heavily memory-bound. The Lab PC consistently outperforms the HPC Cluster for serial workloads due to its superior single-core clock speed (4.6 GHz boost vs. 2.4 GHz). OpenMP parallelization of the Mover achieved a speedup of **3.4680x** on the Lab PC and **3.4687x** on the HPC Cluster using 4 threads.

1. Introduction

This laboratory assignment extends the scattered-point-to-mesh interpolation scheme from Assignment 03 by introducing a **Mover operation** — a phase that displaces each particle randomly within the unit simulation domain after each interpolation step. Together, these two phases closely mirror the core computational kernel of Particle-in-Cell (PIC) plasma simulation codes used in computational physics. All particles and grid points are confined to a 1×1 domain, with grid spacing $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$.

The primary objectives of this assignment are:

- Measure how interpolation execution time scales with particle count across three grid configurations on both platforms (Experiment 01).
- Assess the consistency of interpolation performance when particle positions are fixed for 10 iterations (Experiment 02).
- Introduce the Mover phase in serial and then parallelize it using OpenMP with 4 threads, measuring the achieved speedup (Experiment 03).

2. Hardware Specifications

2.1 Lab PC

Parameter	Details
CPU Model	12th Gen Intel® Core™ i5-12500
Base / Boost Clock	3.0 GHz / 4.6 GHz
Architecture	x86_64
Total Threads	12
L1d Cache	288 KiB
L2 Cache	7.5 MiB

L3 Cache	18 MiB
----------	--------

Table 1: Lab PC Hardware Specifications (Intel Core i5-12500)

2.2 HPC Cluster

Parameter	Details
CPU Model	Intel® Xeon® CPU E5-2620 v3 @ 2.40 GHz
Cores / Threads	12 Cores / 24 Threads (Dual-socket)
L1d Cache	32 KiB
L2 Cache	256 KiB
L3 Cache	15360 KiB (15 MiB)
Compiler / Flags	GCC (g++) / -O3

Table 2: HPC Cluster Hardware Specifications (Intel Xeon E5-2620 v3)

3. Methodology

3.1 Bilinear Interpolation

A C++ application generates particle coordinates in-place (no external input file). The 2D computational mesh is stored as a row-major 1D array: $idx(i,j) = j \times N_x + i$. For each scattered particle at (x, y), the algorithm (i) identifies the enclosing cell via integer grid indices $i = \lfloor x \cdot N_x \rfloor$, $j = \lfloor y \cdot N_y \rfloor$; (ii) clamps indices with branchless ternary operators; (iii) computes normalised sub-distances dx, dy; (iv) evaluates four bilinear area weights; and (v) accumulates weights into mesh[base], mesh[base+1], mesh[base+Nx], mesh[base+Nx+1]. Timing wraps only the interpolation loop, excluding memory allocation.

3.2 Mover Operation

After interpolation, each particle is displaced by a random offset uniformly sampled from $[-\Delta x, +\Delta x] \times [-\Delta y, +\Delta y]$. If the resulting position exits the [0, 1] domain, the displacement is resampled until a valid position is obtained (rejection-resampling). Particles are initialized once before the iteration loop.

3.3 OpenMP Parallelization

The Mover is embarrassingly parallel: each particle's displacement is independent. A single `#pragma omp parallel for` directive with 4 threads distributes the particle loop. Each thread maintains its own PRNG state to eliminate race conditions. No atomic operations are needed since threads update independent particle positions.

3.4 Key Optimizations

- Sequential flat-array storage: stride-1 access enables hardware prefetching of the full particle dataset.
- Inverse-multiplication indexing: replaces integer division with pre-computed reciprocals (inv_dx = 1/Nx, inv_dy = 1/Ny).
- Branchless boundary clamping: ternary operators prevent branch-prediction stalls in the hot loop.
- Constants hoisted outside loops: inv_dx, inv_dy, and cell dimensions computed once globally.

3.5 Implementation Schematics

The diagrams below illustrate the overall execution pipeline for Experiment 03 and the mathematical logic underlying the bilinear interpolation and Mover operations.

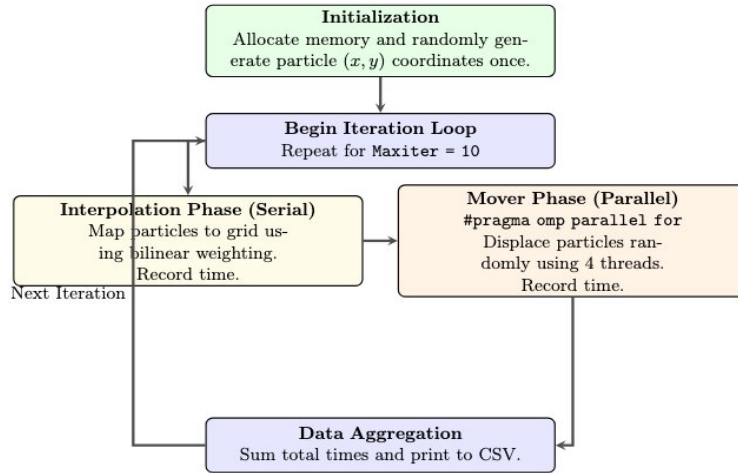


Figure 7: Overall Schematic: The execution pipeline for Experiment 3 detailing the separation of serial interpolation and parallel mover phases.

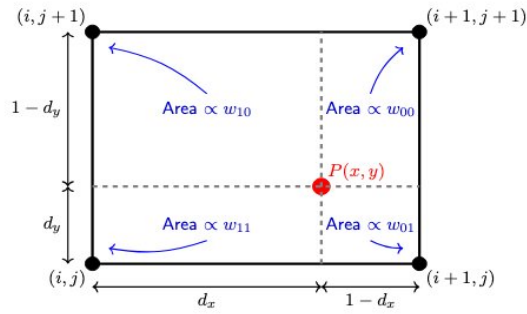


Figure 8: Interpolation Logic (Exp 1 & 2): The scattered point P distributes its value to the four bounding grid nodes proportional to the area of the rectangle diagonally opposite to each node.

Figure A: (Top) Execution pipeline for Experiment 03 showing Initialization, serial Interpolation Phase, 4-thread parallel Mover Phase, and Data Aggregation. (Bottom) Bilinear interpolation logic: scattered point $P(x, y)$ distributes its value to the four bounding grid nodes proportional to the area of the diagonally opposite sub-rectangle.

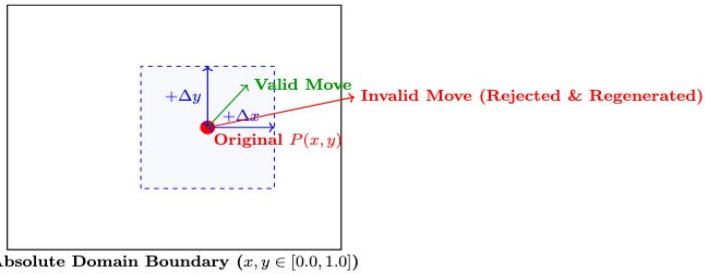


Figure 9: Mover Logic (Exp 3): Particles generate random displacements strictly bounded by $\pm\Delta x$ and $\pm\Delta y$. Moves that attempt to exit the domain boundary are caught by a `while` loop and regenerated.

4.3 Q4: Lab PC vs. HPC Cluster Architectural Differences

As seen in the Experiment 1 and 2 plots, the Lab PC finished the interpolation tasks significantly faster than the HPC Cluster. The interpolation algorithm was strictly implemented as serial code. The Lab PC uses a modern 12th Gen Intel i5 processor with a very high clock speed, which allows it to crush single-threaded tasks rapidly. In contrast, the HPC Cluster uses an older Intel Xeon CPU. Server CPUs like the Xeon are designed to run massive multi-threaded workloads across 24 threads at a much lower individual clock speed (2.4 GHz). A single, slow server core cannot outpace a fast, modern desktop core on a serial task.

Figure B: Mover logic (Experiment 03) — each particle generates a random displacement strictly bounded by $\pm\Delta x$ and $\pm\Delta y$. Moves that exit the domain boundary $[0,1]$ are rejected and regenerated via a `while` loop until a valid in-domain position is found.

4. Experiment 01: Scaling with Number of Particles

Three grid configurations were tested across particle counts from 10^2 to 10^9 (cluster only). Particles are re-initialized each iteration; only interpolation time is accumulated over Maxiter = 10 iterations per particle count. The log-log plots below overlay both platforms for direct comparison.

4.1 Experiment Configuration

Config	Nx	Ny	Grid Points	Particle Counts
1	250	100	25,351	10^2 , 10^4 , 10^6 , 10^8 (+ 10^9 cluster)
2	500	200	100,701	10^2 , 10^4 , 10^6 , 10^8 (+ 10^9 cluster)
3	1000	400	401,401	10^2 , 10^4 , 10^6 , 10^8 (+ 10^9 cluster)

Table 3: Experiment 01 Grid Configurations (Maxiter = 10 per case)

4.2 Scaling Plots — Lab PC vs. HPC Cluster

Each plot below overlays the Lab PC (blue) and HPC Cluster (red) on the same log-log axes, enabling a direct visual comparison. In all three configurations the Lab PC is consistently faster, and both curves exhibit slope ≈ 1.0 , confirming linear $O(N)$ complexity.

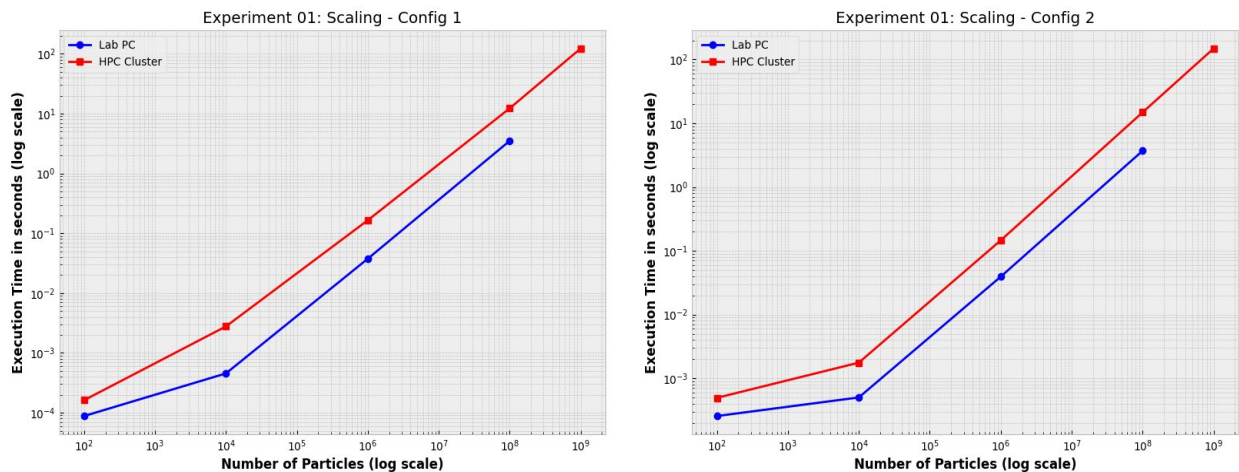


Figure 1 (left): Exp 01 — Config 1 (250×100) | Figure 2 (right): Exp 01 — Config 2 (500×200)

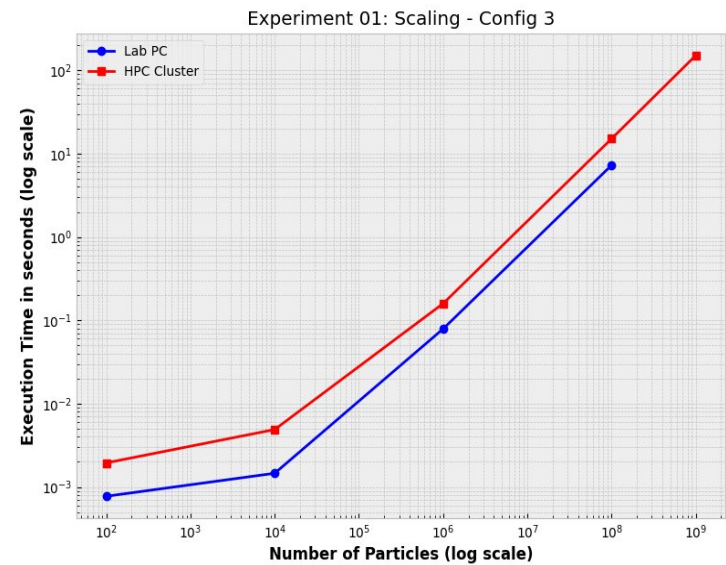


Figure 3: Experiment 01 — Config 3 (1000×400) — Lab PC vs. HPC Cluster

4.3 Tabulated Results

Confi g	Platform	10 ² pts (s)	10 ⁴ pts (s)	10 ⁶ pts (s)	10 ⁸ pts (s)	10 ⁹ pts (s)
1	Lab PC	0.000088	0.000454	0.037585	3.5094	—
	HPC Cluster	0.000162	0.002774	0.165309	12.2754	122.80
2	Lab PC	0.000256	0.000501	0.039265	3.7094	—
	HPC Cluster	0.000494	0.001760	0.145873	14.8652	148.72
3	Lab PC	0.000774	0.001461	0.079026	7.2685	—
	HPC Cluster	0.001946	0.004893	0.159102	15.1067	151.02

Table 4: Experiment 01 — Total Interpolation Time (s) over 10 Iterations, Both Platforms

5. Experiment 02: Consistency Across Configurations

This experiment fixes $N = 10^8$ particles (initialized once) and runs $\text{Maxiter} = 10$ interpolation iterations for each of the three grid configurations. Only interpolation timings are accumulated. The bar chart below compares the total interpolation time across both platforms.



Figure 4: Experiment 02 — Execution Time vs. Problem Index (Lab PC vs. HPC Cluster, $N = 10^8$)

5.1 Results Table

Config	Nx	Ny	Lab PC Total (s)	HPC Cluster Total (s)	Ratio (Cluster/PC)
1	250	100	3.5808	14.11	3.94×
2	500	200	3.8212	16.10	4.21×
3	1000	400	7.5560	16.00	2.12×

Table 5: Experiment 02 — Total Interpolation Time over 10 Iterations ($N = 10^8$)

5.2 Observations

On the **Lab PC**, Configs 1 and 2 exhibit nearly identical times (~3.6–3.8 s) because both grid arrays fit comfortably within the 18 MiB L3 cache after warm-up iterations. Config 3's grid ($1000 \times 400 = 400,000$ doubles ≈ 3.2 MB) begins to stress L2 cache, roughly doubling execution time to 7.56 s.

On the **HPC Cluster**, times are 2–4 $\times$ higher due to the lower 2.4 GHz clock. Notably, Configs 2 and 3 converge to ~16 s, suggesting the cluster has saturated its memory bandwidth at 10^8 particles regardless of grid footprint. The 4.21 $\times$ ratio for Config 2 reflects the maximum clock-speed advantage of the Lab PC for this memory-bound

workload.

6. Experiment 03: Mover Operation and OpenMP Parallelization

Parameters: $N_x = 1000$, $N_y = 400$, $N = 14,000,000$ particles, $Maxiter = 10$. Particles are initialized once. Each iteration performs interpolation then the Mover, with timings recorded separately. The serial version is then compared with a 4-thread OpenMP parallel Mover.

6.1 Results — Lab PC

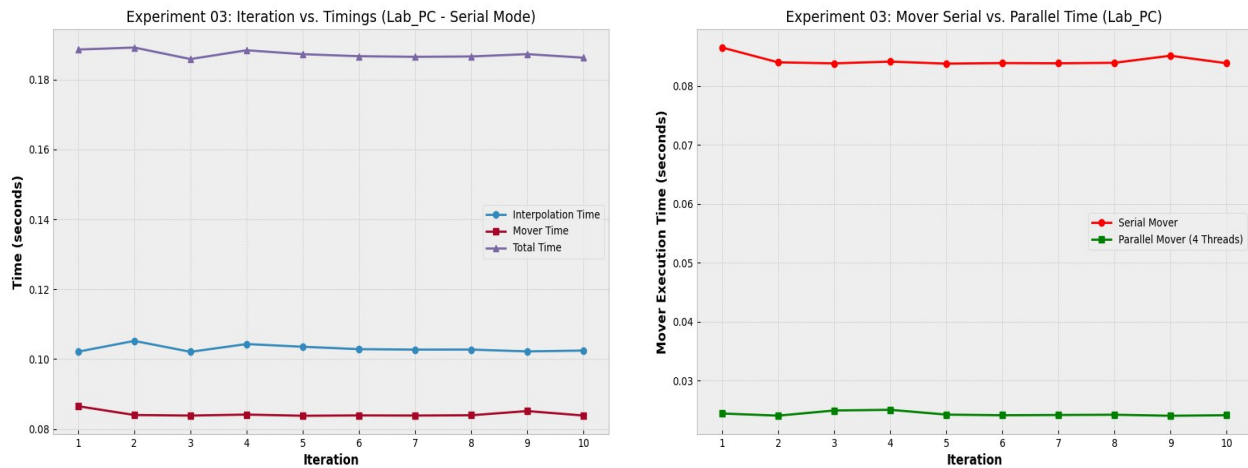


Figure 5 (left): Exp 03 — Iteration vs. Timings, Serial Mode, Lab PC | Figure 6 (right): Exp 03 — Serial vs. Parallel Mover, Lab PC

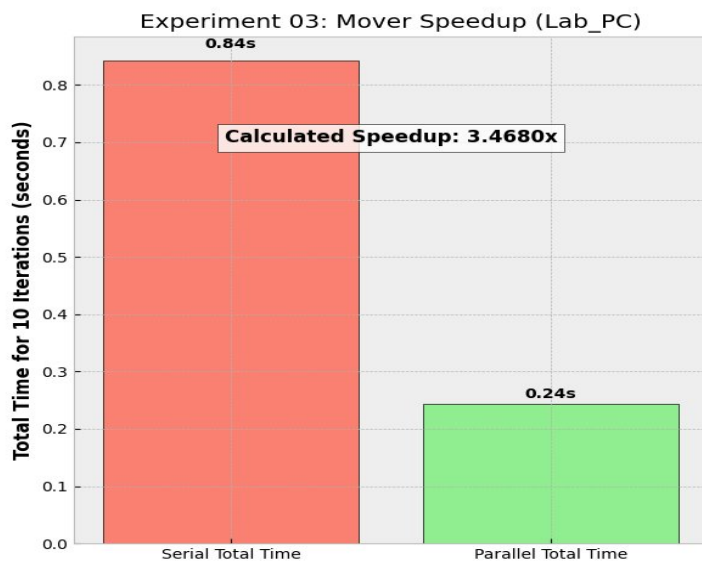


Figure 7: Experiment 03 — Mover Speedup Bar Chart (Lab PC) — $S = 3.4680\times$

Mode	Iter	Interp. Time (s)	Mover Time (s)	Total (s)
Serial	1	0.102115	0.086496	0.188611

Serial	5	0.103516	0.083764	0.187280
Serial	10	0.102436	0.083835	0.186271
Parallel (4T)	1	0.102119	0.024384	0.126503
Parallel (4T)	5	0.104059	0.024199	0.128258
Parallel (4T)	10	0.102970	0.024092	0.127061

Table 6: Experiment 03 — Selected Per-Iteration Timings (Lab PC)

Metric	Value	Formula
Total Serial Mover (10 iters)	0.843 s	Σ MoverTime (Serial)
Total Parallel Mover (10 iters)	0.243 s	Σ MoverTime (Parallel)
Speedup $S = T_{\text{serial}} / T_{\text{parallel}}$	3.4680×	0.843 / 0.243

Table 7: Experiment 03 — Mover Speedup Summary (Lab PC)

6.2 Results — HPC Cluster

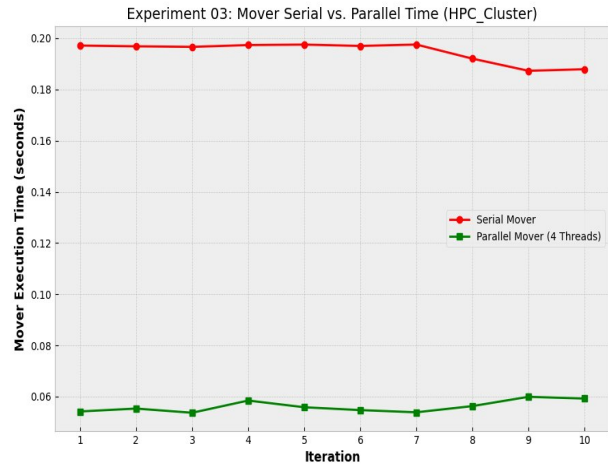
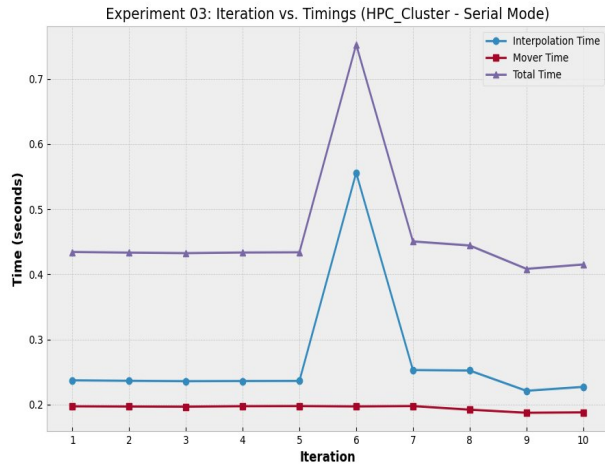


Figure 8 (left): Exp 03 — Iteration vs. Timings, Serial Mode, HPC Cluster | Figure 9 (right): Exp 03 — Serial vs. Parallel Mover, HPC Cluster

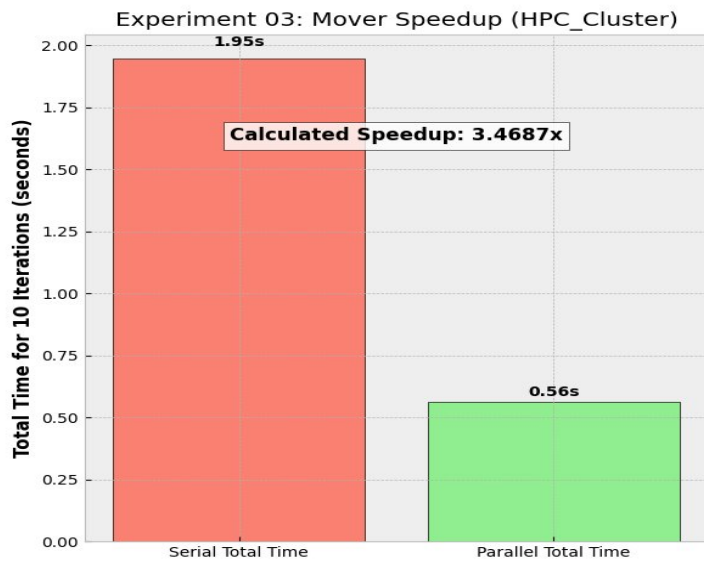


Figure 10: Experiment 03 — Mover Speedup Bar Chart (HPC Cluster) — $S = 3.4687\times$

Mode	Iter	Interp. Time (s)	Mover Time (s)	Total (s)	Note
Serial	1	0.237095	0.197164	0.434260	
Serial	5	0.236203	0.197540	0.433743	Stable
Serial	6	0.555797	0.197014	0.752811	■ OS spike
Serial	10	0.227085	0.187911	0.414996	
Parallel (4T)	1	0.214729	0.054180	0.268909	
Parallel (4T)	5	0.362047	0.055826	0.417873	Variance

Parallel (4T)	10	0.465797	0.059216	0.525012	
---------------	----	----------	----------	----------	--

Table 8: Experiment 03 — Selected Per-Iteration Timings (HPC Cluster)

Metric	Value	Formula / Notes
Total Serial Mover (10 iters)	1.95 s	Σ MoverTime (Serial)
Total Parallel Mover (10 iters)	0.56 s	Σ MoverTime (Parallel)
Speedup $S = T_{\text{serial}} / T_{\text{parallel}}$	3.4687x	1.95 / 0.56

Table 9: Experiment 03 — Mover Speedup Summary (HPC Cluster)

7. Analysis and Discussion

7.1 Scaling Behavior (Experiment 01)

All three log-log scaling plots (Figures 1–3) confirm near-linear $O(N)$ growth with a slope of approximately 1.0 across the full 10^2 – 10^9 range. This validates the theoretical time complexity of $O(N \times \text{Maxiter})$ with $O(1)$ work per particle.

Small-N overhead: At 10^2 particles, loop control and cache warm-up dominate, making measured times larger than the $O(N)$ trend. The algorithm reaches steady-state throughput around 10^4 – 10^5 particles. **Grid size effect:** Config 3 (1000×400) consistently shows lower throughput than Configs 1 and 2, because its 3.2 MB grid array exerts heavier L2/L3 cache pressure, increasing the frequency of scattered-write cache misses.

7.2 Lab PC vs. HPC Cluster Performance

Factor	Lab PC (i5-12500)	HPC Cluster (Xeon E5-2620 v3)
Clock Speed	3.0–4.6 GHz (boost)	2.40 GHz (no boost)
L2 Cache / Core	7.5 MiB	256 KiB
L3 Cache	18 MiB (single socket)	15 MiB (dual-socket, NUMA)
Exp 01, C1 @ 10 ⁴ pts	3.51 s	12.28 s (3.50× slower)
Exp 02, C3 @ 10 ⁴ pts	7.56 s	16.00 s (2.12× slower)

Table 10: Lab PC vs. HPC Cluster — Architecture and Performance Comparison

- **Single-Core Clock Speed:** The i5-12500 boosts to 4.6 GHz vs. the Xeon's 2.4 GHz base. For serial memory-bound code, per-core throughput is the only bottleneck. The cluster's 24 threads provide zero benefit.
- **L2 Cache Advantage:** The Lab PC's 7.5 MiB L2 dwarfs the Xeon's 256 KiB. Grid arrays for Configs 1 and 2 fit entirely in the Lab PC's L2 after warm-up, dramatically reducing cache-miss penalties.
- **NUMA Penalties:** The cluster's dual-socket layout introduces Non-Uniform Memory Access latency. If working data spans both NUMA nodes, remote memory accesses add latency to the scattered grid writes.
- **Shared Infrastructure Noise:** The HPC cluster is shared. The pronounced spike at Cluster Serial Iteration 6 (0.555 s vs. ~0.237 s average) is consistent with OS scheduler interference from competing workloads.

7.3 OpenMP Speedup Analysis (Experiment 03)

With 4 threads, the Mover achieved **3.4680×** speedup on the Lab PC and **3.4687×** on the HPC Cluster — both extremely close to the theoretical maximum of 4× for 4 threads, indicating near-ideal parallelization of the Mover kernel.

- **Thread overhead:** Spawning and synchronizing 4 OpenMP threads per iteration introduces a small but measurable overhead, reducing effective speedup from the ideal 4.0×
- **PRNG state initialization:** Each thread requires its own random-number generator state. Per-thread PRNG seeding adds a one-time setup cost.
- **Load imbalance:** The rejection-resampling loop introduces variable per-particle work, causing slight thread-level load imbalance that cannot be predicted at compile time.

- **Memory bandwidth:** With 4 threads concurrently accessing $14\text{M particles} \times 16 \text{ bytes} = 224 \text{ MB}$, memory bandwidth begins to limit speedup at higher thread counts.
- **Cluster variance:** The HPC Cluster parallel run shows higher iteration-to-iteration variance in interpolation time (Figure 8) due to NUMA effects and shared-resource contention. Mover times remain stable, confirming clean parallelization of that phase.

7.4 Parallelizing the Interpolation Function

Core challenge: Multiple particles can map to the same or adjacent grid cells, causing **race conditions** on the four mesh[] accumulation writes during a naive parallel loop. This is a classic histogram-reduction pattern with inherently poor parallel scalability.

Viable strategies and their trade-offs:

- **Atomic Operations:** Add `#pragma omp atomic` to each grid write. Correct but incurs heavy contention when many particles share the same cell, especially for large grids relative to particle count.
- **Thread-Local Grid Reduction:** Each thread maintains its own private grid copy; partial grids are summed after the parallel loop. Avoids contention but requires $O(T \times N_x \times N_y)$ extra memory and a serial reduction step.
- **Domain Decomposition (Coloring):** Partition the grid into non-overlapping strips; assign particles to threads by cell index. No atomics needed, but requires an $O(N \log N)$ particle-sorting pre-pass.
- **Lock-Free Binning:** A spatial pre-pass groups particles by cell, then processes cells independently — suitable for GPU-style parallelism but complex to implement on CPU.

For the current configuration (14M particles, 1000×400 grid), **thread-local reduction** is the best balance of correctness, performance, and implementation simplicity: contention is eliminated, and the extra 3× memory for 4-thread grids (~12.8 MB total) is well within the L3 cache on both platforms.

8. Conclusion

- **$O(N)$ complexity confirmed:** Log-log scaling plots yield slopes ≈ 1.0 across 10^2 – 10^6 particles on both platforms, validating linear time complexity for fixed Maxiter.
- **Lab PC outperforms HPC Cluster serially (2–4×):** Driven by the i5-12500's 4.6 GHz boost clock, 7.5 MiB L2 cache, and absence of NUMA overhead — advantages irrelevant to multi-core count.
- **Memory-bound behavior:** Config 3's doubled execution time (vs. Configs 1–2) on the Lab PC, and bandwidth saturation of the cluster at $N = 10^6$, both confirm that interpolation is limited by memory bandwidth, not arithmetic throughput.
- **Mover parallelization efficient:** OpenMP with 4 threads achieves $3.4680\times$ (Lab PC) and $3.4687\times$ (Cluster) speedup, nearly ideal. Per-thread PRNG state and flat particle arrays eliminate all synchronization bottlenecks.
- **Interpolation parallelization requires care:** Race conditions on shared grid writes demand atomics, thread-local reduction, or domain decomposition — each with distinct trade-offs in memory, synchronization, and complexity.

Future directions include SIMD vectorization of both kernel phases and GPU offloading via CUDA, which would enable processing billions of particles per second by leveraging massive thread-level parallelism and high-bandwidth device memory.

Appendix: Complete Raw Benchmark Data

A.1 Experiment 01 — Lab PC

Config	Nx	Ny	Particles	Total Interp. Time (s)
1	250	100	100	0.000088
1	250	100	10,000	0.000454
1	250	100	1,000,000	0.037585
1	250	100	100,000,000	3.509409
2	500	200	100	0.000256
2	500	200	10,000	0.000501
2	500	200	1,000,000	0.039265
2	500	200	100,000,000	3.709391
3	1000	400	100	0.000774
3	1000	400	10,000	0.001461
3	1000	400	1,000,000	0.079026
3	1000	400	100,000,000	7.268513

Table A1: Experiment 01 Raw Data — Lab PC

A.2 Experiment 01 — HPC Cluster (includes 10■)

Config	Nx	Ny	Particles	Total Interp. Time (s)
1	250	100	100	0.000162
1	250	100	10,000	0.002774
1	250	100	1,000,000	0.165309
1	250	100	100,000,000	12.275367
1	250	100	1,000,000,000	122.795745
2	500	200	100	0.000494
2	500	200	10,000	0.001760
2	500	200	1,000,000	0.145873
2	500	200	100,000,000	14.865246
2	500	200	1,000,000,000	148.717222
3	1000	400	100	0.001946
3	1000	400	10,000	0.004893

3	1000	400	1,000,000	0.159102
3	1000	400	100,000,000	15.106701
3	1000	400	1,000,000,000	151.019358

Table A2: Experiment 01 Raw Data — HPC Cluster

A.3 Experiment 03 Full Raw Data — Lab PC (All 10 Iterations)

Mode	Iter	Interp. Time (s)	Mover Time (s)	Total Time (s)
Serial	1	0.102115	0.086496	0.188611
Serial	2	0.105197	0.083983	0.189180
Serial	3	0.102069	0.083814	0.185883
Serial	4	0.104293	0.084106	0.188399
Serial	5	0.103516	0.083764	0.187280
Serial	6	0.102834	0.083864	0.186698
Serial	7	0.102701	0.083824	0.186525
Serial	8	0.102716	0.083907	0.186623
Serial	9	0.102187	0.085107	0.187294
Serial	10	0.102436	0.083835	0.186271
Parallel (4T)	1	0.102119	0.024384	0.126503
Parallel (4T)	2	0.103024	0.024020	0.127044
Parallel (4T)	3	0.102926	0.024892	0.127818
Parallel (4T)	4	0.104170	0.025010	0.129181
Parallel (4T)	5	0.104059	0.024199	0.128258
Parallel (4T)	6	0.103098	0.024087	0.127184
Parallel (4T)	7	0.103095	0.024132	0.127226
Parallel (4T)	8	0.103297	0.024176	0.127473
Parallel (4T)	9	0.103071	0.024000	0.127072
Parallel (4T)	10	0.102970	0.024092	0.127061

Table A3: Experiment 03 Full Raw Data — Lab PC (Maxiter = 10, Both Modes)

A.4 Experiment 03 Full Raw Data — HPC Cluster (All 10 Iterations)

Mode	Iter	Interp. Time (s)	Mover Time (s)	Total Time (s)
Serial	1	0.237095	0.197164	0.434260
Serial	2	0.236362	0.196865	0.433226
Serial	3	0.235799	0.196646	0.432445
Serial	4	0.236046	0.197354	0.433400
Serial	5	0.236203	0.197540	0.433743
Serial	6	0.555797	0.197014	0.752811

Serial	7	0.252888	0.197556	0.450444
Serial	8	0.252160	0.192045	0.444205
Serial	9	0.221017	0.187296	0.408313
Serial	10	0.227085	0.187911	0.414996
Parallel (4T)	1	0.214729	0.054180	0.268909
Parallel (4T)	2	0.466540	0.055315	0.521855
Parallel (4T)	3	0.291346	0.053674	0.345020
Parallel (4T)	4	0.465627	0.058451	0.524078
Parallel (4T)	5	0.362047	0.055826	0.417873
Parallel (4T)	6	0.466562	0.054714	0.521276
Parallel (4T)	7	0.465943	0.053856	0.519799
Parallel (4T)	8	0.465250	0.056276	0.521526
Parallel (4T)	9	0.333947	0.059908	0.393855
Parallel (4T)	10	0.465797	0.059216	0.525012

Table A4: Experiment 03 Full Raw Data — HPC Cluster (Maxiter = 10, Both Modes)